CATEGORY C — PART A: CODING PROBLEMS

Problem 1: Vowel and Consonant Counter

TASK

Write a program that reads a single word and then counts and prints the total number of vowels and consonants in that word. Assume the input word contains only alphabetic characters (case-insensitive).

EXAMPLE

Sample Input: Programming
Expected Output:
Vowels: 3
Consonants: 8

HINT

Iterate through each character of the word. Use conditional statements to check if a character is a vowel ('a', 'e', 'i', 'o', 'u'). All other alphabetic characters are consonants.

SOLUTION (JAVA)

```
import java.util.Scanner; public class VowelConsonantCounter { public static void main(String[] args) { Scanner sc = new Scanner(System.in); String word = sc.next().toLowerCase(); int vowels = 0, consonants = 0; for (int i = 0; i < word.length(); i++) { char ch = word.charAt(i); if ("aeiou".indexOf(ch) != -1) vowels++; else if (Character.isLetter(ch)) consonants++; } System.out.println("Vowels: " + vowels); System.out.println("Consonants: " + consonants); } }
```

Problem 2: Word Reverser and Palindrome Checker

TASK

Create a program that reads a word, reverses it using a mutable text builder (like StringBuilder in Java), and prints the reversed word. Determine and print whether the word is a palindrome.

EXAMPLE

Sample Input: level | java
Expected Output:
level | avaj
palindrome | not a palindrome

HINT

Use a mutable string class to build the reversed word. Compare the original string with the reversed string to check for palindrome property.

SOLUTION (JAVA)

```
import java.util.Scanner; public class WordReverser { public static void main(String[] args) { Scanner sc = new Scanner(System.in); String word = sc.next(); StringBuilder sb = new StringBuilder(word); String reversed = sb.reverse().toString(); System.out.println(reversed); if (word.equalsIgnoreCase(reversed)) { System.out.println("palindrome"); } else { System.out.println("not a palindrome"); } } }
```

Problem 3: Even and Odd Numbers in a List

TASK

Given a list of integers, write a program to count how many numbers in the list are even and how many are odd. Print the counts for both categories.

EXAMPLE

Sample Input: [3, 8, 12, 5, 7, 10]
Expected Output: Even: 3 | Odd: 3

HINT

Iterate through each number in the list. Use the modulo operator (%) to determine if a number is even or odd.

PROBLEM 3 SOLUTION (JAVA)

```
public class EvenOddCounter { public static void main(String[] args) { int[] nums = {3, 8, 12, 5, 7, 10}; int even = 0, odd = 0; for (int n : nums) { if (n % 2 == 0) even++; else odd++; } System.out.println("Even: " + even); System.out.println("Odd: " + odd); } }
```

Problem 4: In-Place Array Reversal

TASK

Write a program that reverses the order of token numbers stored in a list in-place (modify original list directly).

EXAMPLE

Sample Input: [11, 22, 33, 44] | **Expected Output:** [44, 33, 22, 11]

HINT

Use two pointers starting from beginning and end, swapping elements until they meet or cross.

SOLUTION (JAVA)

```
import java.util.Arrays; public class ArrayReversal { public static void main(String[] args) { int[] arr = {11, 22, 33, 44}; int left = 0, right = arr.length - 1; while (left < right) { int temp = arr[left]; arr[left] = arr[right]; arr[right] = temp; left++; right--; } System.out.println(Arrays.toString(arr)); } }
```

Problem 5: Digit Sum and Number Reversal

TASK

Calculate and print the sum of digits of a positive integer, and print the number with its digits reversed.

PROBLEM 5 EXAMPLE & HINT

Sample Input: 4721 | **Expected Output:** Sum of digits: 14, Reverse: 1274

Hint: Use modulo (%) by 10 to extract digits and integer division (/) by 10 to remove digits.

PROBLEM 5 SOLUTION (JAVA)

```
public class DigitSumReverse { public static void main(String[] args) { int n = 4721, sum = 0, rev = 0; while (n > 0) { int digit = n % 10; sum += digit; rev = rev * 10 + digit; n /= 10; } System.out.println("Sum of digits: " + sum); System.out.println("Reverse: " + rev); } }
```

Problem 6: Prime Number Checker

TASK

Write a program that reads a number greater than 1 and determines whether it is a prime number.

EXAMPLE

Sample Input: 29 / 21 | **Expected Output:** 29 is prime / 21 is not prime

HINT

Iterate from 2 up to the square root of the number. If any number divides it evenly, it is not prime.

SOLUTION (JAVA)

```
public class PrimeChecker { public static void checkPrime(int n) { boolean isPrime = n > 1; for (int i = 2; i * i <= n; i++) { if (n % i == 0) { isPrime = false; break; } } System.out.println(n + (isPrime ? " is prime" : " is not prime")); } public static void main(String[] args) { checkPrime(29); checkPrime(21); } }
```

Problem 7: Factorial Calculation

TASK

Write a program that reads a non-negative integer and calculates its factorial (0! = 1).

EXAMPLE

Sample Input: 5 / 0 | **Expected Output:** 120 / 1

SOLUTION (JAVA)

```
public class Factorial { public static long getFactorial(int n) { long fact = 1; for (int i = 1; i <= n; i++) fact *= i; return fact; } public static void main(String[] args) { System.out.println(getFactorial(5)); System.out.println(getFactorial(0)); } }
```

Problem 8: Student Grade Assignment

TASK

Assign grades based on marks: A (≥90), B (75-89), C (60-74), D (40-59), F (<40).

EXAMPLE

Sample Input: 72 / 95 / 35 | **Expected Output:** Grade C / Grade A / Grade F

SOLUTION (JAVA)

```
public class GradeAssigner { public static void printGrade(int marks) { if (marks >= 90) System.out.println("Grade A"); else if (marks >= 75) System.out.println("Grade B"); else if (marks >= 60) System.out.println("Grade C"); else if (marks >= 40) System.out.println("Grade D"); else System.out.println("Grade F"); } public static void main(String[] args) { printGrade(72); printGrade(95); printGrade(35); } }
```

Problem 9: Leap Year Detector

TASK

Determine if a year is a leap year (divisible by 4, not by 100 unless divisible by 400).

EXAMPLE

Sample Input: 2024 / 1900 / 2000 | **Expected Output:** Leap year / Not a leap year / Leap year

SOLUTION (JAVA)

```
public class LeapYear { public static void checkLeap(int year) { boolean isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0); System.out.println(isLeap ? "Leap year" : "Not a leap year"); } public static void main(String[] args) { checkLeap(2024); checkLeap(1900); checkLeap(2000); } }
```

Problem 10: Student Result Card Generator

TASK

Design a Student class with name and 3 marks. Calculate average and print name in uppercase, average (1 decimal place), and grade ('B' for ≥75, 'D' for 40-59, etc.).

SOLUTION (JAVA)

```
class Student { String name; int[] marks; public Student(String name, int[] marks) { this.name = name; this.marks = marks; } public double getAverage() { return (marks[0] + marks[1] + marks[2]) / 3.0; } public char getGrade() { double avg = getAverage(); if (avg >= 75) return 'B'; if (avg >= 60) return 'C'; if (avg >= 40) return 'D'; return 'F'; } public void printCard() { System.out.printf("%s: Average %.1f, Grade %c\n", name.toUpperCase(), getAverage(), getGrade()); } }
```

PART B — QUIZ

Q1. Consider two String objects, s1 and s2:

```
String s1 = new String("hello");
String s2 = new String("hello");
```

Which statement is true regarding comparison of s1 and s2?

A. s1==s2 is true, s1.equals(s2) is true.
B. s1==s2 is false, s1.equals(s2) is false.
C. s1==s2 is true, s1.equals(s2) is false.
D. s1==s2 is false, s1.equals(s2) is true.

ANSWER: D (s1==s2 compares object references; equals compares values)

Q2. Given operations on StringBuilder:

```
sb.append("Java"); sb.append(" Lab"); sb.reverse();
```

What will be the content of sb?

A. "Java Lab" B. "bal avaJ" C. "Lab Java" D. "aval baJ"

ANSWER: B ("bal avaJ")

Q3. For Student constructor overload execution:

```
Student s = new Student("Ravi");
```

Which constructor will be directly called?

A. public Student()
B. public Student(String name)
C. public Student(String name, String department)
D. Compile-time error due to ambiguity

ANSWER: B (public Student(String name))

Q4. Wallet reference variables execution:

`Wallet w1 = new Wallet(500); Wallet w2 = w1; w2.add(200);`
What will be the balance seen through w1 and w2?

A. w1 balance: 500, w2 balance: 700
B. w1 balance: 700, w2 balance: 500
C. w1 balance: 500, w2 balance: 500
D. w1 balance: 700, w2 balance: 700

ANSWER: D (w1 and w2 reference the exact same object in heap memory)

Q5. Visitor class tracking total visitors using static variable count:

Creating three instances: `new Visitor("Asha"); new Visitor("Ravi"); new Visitor("Neha");`
What will be returned by `Visitor.getTotalVisitors()`?

A. 0 B. 1 C. 3 D. Undefined

ANSWER: C (3, static counter is shared across all instances)

Q6. Array iteration output:

`int[] scores = {45, 78, 92, 60, 85};` iterating index 0 to scores.length - 1:

A. 45 78 92 60 85 B. 85 60 92 78 45 C. 45 78 92 60 D. Error

ANSWER: A (45 78 92 60 85)

Q7. For loop with continue:

```
for(int i=1; i<=5; i++) { if(i==3) continue; System.out.print(i+" "); }
```

A. 1 2 3 4 5 B. 1 2 4 5 C. 3 D. 1 2

ANSWER: B (1 2 4 5)

Q8. Pass-by-value primitive method:

`int x = 5; change(x); System.out.println(x);` where `change(int n) { n = 10; }`

A. 5 B. 10 C. 0 D. Error

ANSWER: A (5, primitive arguments are passed by value)

Q9. Division operation:

```
int total = 250; double count = 3; double avg = total / count;
```

A. 83 B. 83.0 C. 83.33333333333333 D. 83.33

ANSWER: C (83.33333333333333)

Q10. Abstract class & Polymorphism:

`Payment p = new Card(); p.pay(500);` (Card overrides pay with 2% fee)

A. Paid 500.0 by card B. Paid 510.0 by card C. Error

ANSWER: B (Paid 510.0 by card)

Q11. String toUpperCase() immutability state:

```
String name = "java"; String upperName = name.toUpperCase();
```

A. name changes to "JAVA"
B. name remains "java", new String "JAVA" created for upperName
C. Both refer to same new String
D. Original destroyed

ANSWER: B (name remains "java", new String object "JAVA" created)

Q12. String objects count in memory:

```
String s1 = "hello"; String s2 = "hello"; String s3 = new String("hello");
```

A. 1 B. 2 C. 3 D. 0

ANSWER: B (2 objects: 1 in String pool shared by s1 & s2, 1 in Heap for s3)

Q13. Word length and charAt(2) output for "Campus":

A. 6 / m B. 6 / p C. 5 / m D. 7 / p

ANSWER: A (6 length, 'm' at index 2)

Q14. email.indexOf("@") for "user@domain.com":

A. 4 B. 0 C. 5 D. -1

ANSWER: A (4)

Q15. Default constructor initialization for Book:

```
Book b = new Book(); System.out.println(b.getDetails());
```

A. Title: null, Price: 0.0 B. Title: Unknown, Price: 0.0 C. Compilation Error

ANSWER: B (Title: Unknown, Price: 0.0)

Q16. Parameterized constructor for Book:

```
Book b = new Book("Java Programming", 450.0);
```

A. Title: Java Programming, Price: 450.0 B. Title: null, Price: 0.0

ANSWER: A (Title: Java Programming, Price: 450.0)

Q17. Objects count in memory:

```
Wallet w1 = new Wallet(500); Wallet w2 = w1; Wallet w3 = new Wallet(500);
```

A. 1 B. 2 C. 3 D. 0

ANSWER: B (2 distinct object instances created via 'new')

Q18. Wallet balance after w2.add(200):

A. w1 balance: 500 / w2 balance: 700
B. w1 balance: 700 / w2 balance: 500
C. w1 balance: 700 / w2 balance: 700

ANSWER: C (w1 balance: 700 / w2 balance: 700)

Q19. Instance method setName execution:

```
Student s1 = new Student("Asha"); Student s2 = new Student("Ravi");
s1.setName("Anu");
```

A. s1 name: Asha / s2 name: Ravi B. s1 name: Anu / s2 name: Anu
C. s1 name: Anu / s2 name: Ravi D. s1 name: Asha / s2 name: Anu

ANSWER: C (s1 name: Anu / s2 name: Ravi)

Q20. Static visitorCount variable tracking:

```
Visitor v1 = new Visitor(); Visitor v2 = new Visitor(); Visitor v3 = new
Visitor();
```

A. Total visitors: 0 B. Total visitors: 1 C. Total visitors: 3

ANSWER: C (Total visitors: 3)

PART C — CONCEPT QUESTIONS

Detailed responses to theoretical and core Java concept questions are provided on subsequent pages.

PART C — CONCEPT QUESTIONS (1 TO 5)

Q1. String Immutability and Performance:
String immutability means once a String object is created, its value cannot be modified. Any modification creates a new String instance in heap memory. In loops with frequent modifications, this creates excessive temporary objects, degrading performance and increasing Garbage Collection overhead. This is mitigated by using mutable classes like `StringBuilder` or `StringBuffer`.

Q2. Resolving Ambiguity with 'this' Keyword:
When constructor parameters share names with instance variables, parameter names shadow fields. The `this` keyword explicitly references the current class instance variable (e.g., `this.name = name;`), resolving variable shadowing ambiguity.

Q3. Object vs. Reference Variable:
An object is data allocated in Heap memory containing state and behavior. A reference variable is stored on the Stack and holds the memory address pointing to that object. Assigning one reference to another (`w2 = w1`) copies the address so both references point to the same object.

Q4. Static vs. Instance Variables:
Static variables belong to the class level and share state across all instances (e.g., total visitor count in a venue). Instance variables belong to specific objects and maintain unique states per instance (e.g., individual bank account balance).

Q5. Comparison of Loops (for, while, do-while):
A `for` loop is preferred when iteration count is known beforehand (e.g., traversing arrays). A `while` loop is condition-controlled and checked before execution (e.g., reading stream data until EOF). A `do-while` loop executes at least once before testing condition (e.g., interactive CLI menu prompts).

PART C — CONCEPT QUESTIONS (6 TO 10)

Q6. Pass-by-Value Mechanism in Java:
Java is strictly pass-by-value. For primitive types, a copy of the value is passed, so changes inside methods do not affect the original variable. For object references, a copy of the memory address reference is passed; modifying fields via that reference mutates the underlying heap object, but reassigning the reference variable itself doesn't affect caller's reference.

Q7. Integer Division and Type Casting:
Integer division truncates fractional components and returns an integer quotient (e.g., 5 / 2 = 2). Explicit casting of at least one operand to double (e.g., `(double) 5 / 2`) promotes arithmetic evaluation to floating-point division, returning precise decimal results (2.5).

Q8. Encapsulation & Access Control:
Encapsulation wraps data and implementation details inside a class while protecting direct external access. Marking variables `private` and providing `public` getters and setters enforces data validation, maintains class invariants, and guards internal state integrity.

Q9. Operator '==' vs. 'equals()' Method:
The == operator compares object memory references to check if two variables point to identical memory addresses. The `equals()` method evaluates logical equality of content when overridden by classes like `String`.

Q10. Method Overloading vs. Method Overriding:
Method overloading defines multiple methods in the same class with same name but different signatures, resolved at compile-time (static polymorphism). Method overriding allows a subclass to provide a specific implementation of a parent class method with identical signature, resolved at run-time (dynamic polymorphism).